8

ПРОЕКТИРОВАНИЕ СИСТЕМЫ ДЛЯ СОКРАЩЕНИЯ URL-АДРЕСОВ

В этой главе мы рассмотрим интересную классическую задачу, с которой можно столкнуться на интервью по проектировании ИТ-систем: разработка сервиса для сокращения URL-адресов по примеру tinyurl.

ШАГ 1: ПОНЯТЬ ЗАДАЧУ И ОПРЕДЕЛИТЬ МАСШТАБ РЕШЕНИЯ

На интервью по проектированию ИТ-систем специально дают задачи, допускающие свободную трактовку. Чтобы разработать хорошо продуманную систему, необходимо задавать уточняющие вопросы.

Кандидат: «Можете ли привести пример работы сервиса для сокращения URL-адресов?»

Интервьюер: «Допустим, https://www.systeminterview.com/q=chatsystem&c=loggedin&v=v3&l=long — это исходный URL-адрес. Ваш сервис должен создать ссылку покороче: https://tinyurl.com/y7keocwj. Клик по этой ссылке должен перенаправлять к исходному URL-адресу».

Кандидат: «Какой объем трафика?»

Интервьюер: «100 миллионов сгенерированных URL-адресов в день».

Кандидат: «Какая длина должна быть у сокращенного URL-адреса?»

Интервьюер: «Как можно короче».

Кандидат: «Какие символы допускаются в сокращенном URL-адресе?»

Интервьюер: «Сокращенный URL-адрес может содержать цифры (0–9) и буквы (a–z, A–Z)».

Кандидат: «Допускается ли удаление или обновление сокращенного URL-адреса?»

Интервьюер: «Чтобы не усложнять, предположим, что сокращенный URL-адрес не подлежит удалению или обновлению».

Вот типичные сценарии использования:

1. Сокращение URL-адресов: дается длинный URL-адрес => возвращается намного более короткий URL-адрес.
2. Перенаправление URL-адресов: дается сокращенный URL-адрес => пользователь перенаправляется к исходному URL-адресу.
3. Высокая доступность, масштабируемость и устойчивость к сбоям.

Приблизительные оценки

- Операции записи: генерируется 100 миллионов URL-адресов в день.
- Операций записи в секунду: 100 миллионов / 24 / 3600 = 1160.
- Операции чтения: если предположить, что операции чтения и записи имеют соотношение 10 к 1, за одну секунду будет выполняться 1160 * 10 = 11 600 операций чтения.
- Если предположить, что сервис для сокращения URL-адресов проработает 10 лет, мы должны поддерживать хранение 100 миллионов * 365 * 10 = 365 миллиардов записей.
- Пусть длина среднего URL-адреса составляет 100 символов.
- Требования к хранилищу в ближайшие 10 лет: 365 миллиардов * 100 байт * 10 лет = 365 Тб.

Вы должны обсудить эти предположения и расчеты с интервьюером и убедиться в том, что вы поняли друг друга правильно.

ШАГ 2: ПРЕДЛОЖИТЬ ОБЩЕЕ РЕШЕНИЕ И ПОЛУЧИТЬ СОГЛАСИЕ

В этом разделе мы обсудим конечные точки API, перенаправление и сокращение URL-адресов.

Конечные точки API

Конечные точки API обеспечивают взаимодействие между клиентами и серверами. Наш API будет в стиле REST. Если вы не знакомы с этим стилем, можете обратиться к справочным материалам (например, [1]). Сервису сокращения URL-адресов нужны две основные конечные точки:

1. Сокращение URL-адреса. Чтобы создать новый сокращенный URL-адрес, клиент отправляет POST-запрос с одним параметром: исходным длинным URL-адресом. Конечная точка выглядит так:

   `POST api/v1/data/shorten`

   - параметр запроса: `{longUrl: longURLString}`;
   - возвращается `shortURL`.

2. Перенаправление URL-адреса. Чтобы перенаправить сокращенную ссылку к соответствующему длинному URL-адресу, клиент отправляет GET-запрос. Конечная точка выглядит так:

   `GET api/v1/shortUrl`

   - возвращается longURL для HTTP-перенаправления.

Перенаправление URL-адресов

На рис. 8.1 показано, что происходит при вводе в браузере URL-адреса tinyurl. Получив запрос, сервер меняет короткий URL-адрес на длинный, используя перенаправление с кодом 301.

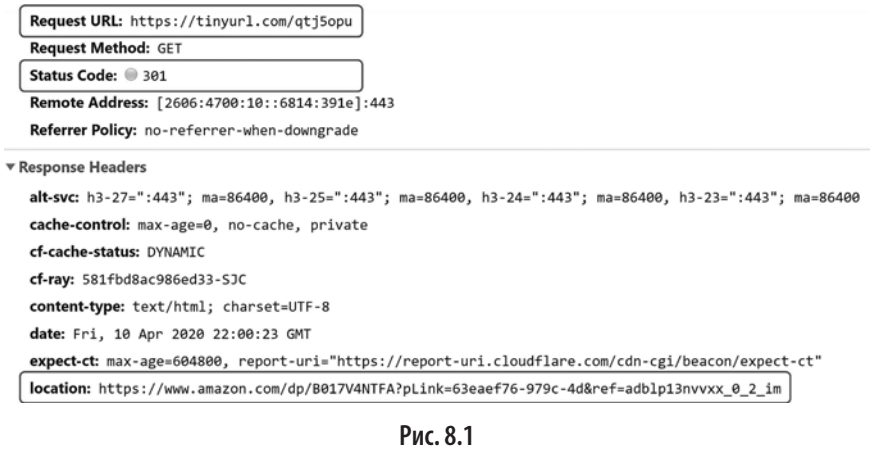


Рис. 8.1

Подробное взаимодействие между клиентами и серверами показано на рис. 8.2.

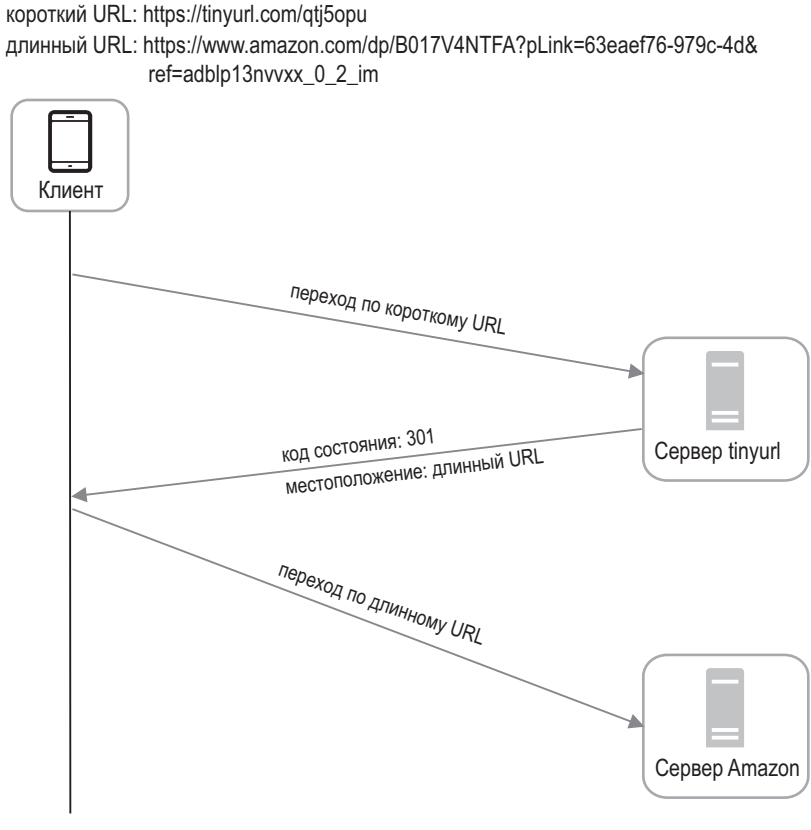


Рис. 8.2

Здесь стоит обсудить отличия между кодами перенаправления 301 и 302.

- **Перенаправление 301**. Код состояния 301 означает, что запрошенный URL-адрес «навсегда» перемещен по длинному URL-адресу. Так как перенаправление постоянное, браузер кэширует ответ и последующие запросы по тому же адресу не будут направляться к нашему сервису. Вместо этого браузер сразу откроет сокращенный URL-адрес.
- **Перенаправление 302**. Код состояния 302 означает, что URL-адрес «временно» перемещен по длинному URL-адресу. То есть после-

дующие запросы того же URL-адреса будут сначала отправляться нашему сервису, а затем перенаправляться к серверу длинного URL-адреса.

У обоих методов перенаправления есть свои плюсы и минусы. Если нам в первую очередь нужно снизить нагрузку на сервер, лучше использовать код состояния 301, так как для каждого сокращенного URL-адреса сервис будет получать только первый запрос. Если же нам нужна аналитическая информация, стоит выбрать код состояния 302, так как он упрощает отслеживание частоты и источника переходов по ссылке.

Самый очевидный способ реализации перенаправления URL-адресов заключается в использовании хеш-таблиц. Если предположить, что хеш-таблица хранит пары `<shortURL, longURL>`, перенаправление можно организовать следующим образом:

- получаем `longURL: longURL = hashTable.get(shortURL);`
- получив `longURL`, выполняем перенаправление.

Сокращение URL-адресов

Допустим, сокращенный URL-адрес выглядит как www.tinyurl.com/{hashValue}. Чтобы его получить, мы должны найти функцию *fx*, которая привязывает длинный URL-адрес к `hashValue`, как показано на рис. 8.3.

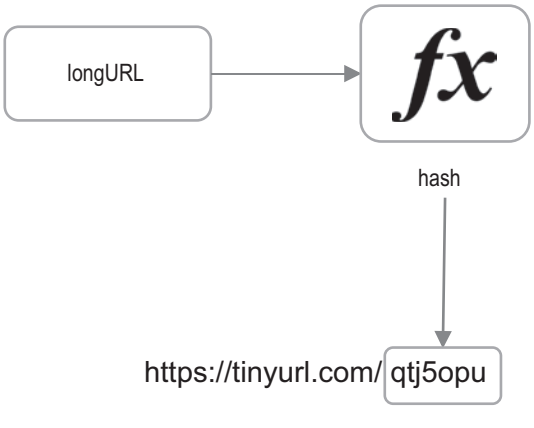


Рис. 8.3

К функции хеширования предъявляются следующие требования:

- каждое значение `longURL` должно иметь один хеш `hashValue`;
- каждое значение `hashValue` должно указывать обратно на `longURL`.

Подробная архитектура функции хеширования обсуждается в следующем разделе.

ШАГ 3: ПОДРОБНОЕ ПРОЕКТИРОВАНИЕ

До сих пор мы обсуждали общие вопросы проектирования сервиса для сокращения и перенаправления URL-адресов. Здесь же мы подробно рассмотрим модель данных, функцию хеширования и процессы сокращения/перенаправления.

Модель данных

При обсуждении общей архитектуры мы решили хранить все в хеш-таблице. Это хорошая отправная точка, но в реальных системах такой подход непрактичен, поскольку ресурсы памяти ограничены и дороги. Пары `<shortURL, longURL>` лучше хранить в реляционной базе данных. На рис. 8.4 показана схема простой таблицы БД, состоящей из трех столбцов: `id`, `shortURL`, `longURL`.

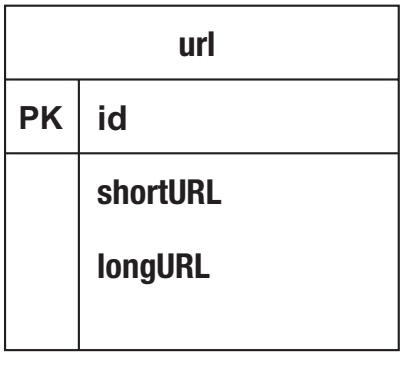


Рис. 8.4

Функция хеширования

Функция хеширования используется для получения из длинного URL-адреса хеша — `hashValue`.

Длина hashValue

`hashValue` состоит из символов [0-9, a-z, A-Z], число которых равно 10 + 26 + + 26 = 62. Чтобы определить длину `hashValue`, нужно найти наименьшее n, при котором $62^n \geq 365$ миллиардов. Судя по этим прикидкам, система должна поддерживать до 365 миллиардов URL-адресов. В табл. 8.1 показаны разные значения длины `hashValue` и соответствующее максимальное количество поддерживаемых URL-адресов.

Таблица 8.1

n	Максимальное количество URL-адресов
1	62^1 = 62
2	62^2 = 3 844
3	62^3 = 238 328
4	62^ 4 = 14 776 336
5	62^5 = 916 132 832
6	62^6 = 56 800 235 584
7	62^7 = 3 521 614 606 208 ~3,5 триллиона
8	62^8 = 218 340 105 584 896

3,5 триллиона (когда n = 62 ^ 7) более чем достаточно для хранения 365 миллиардов URL-адресов, поэтому длина `hashValue` будет равна 7.

Мы исследуем два вида функций хеширования для сокращения URL-адресов: «хеш + разрешение конфликтов» и «преобразование base62».

Хеш + разрешение конфликтов

Чтобы сократить длинный URL-адрес, нужно реализовать хеш-функцию, которая хеширует его в строку из 7 символов. Очевидное решение состоит

в использовании общеизвестных функций хеширования вроде `CRC32`, `MD5` или `SHA-1`. В следующей таблице приводится сравнение хешей, полученных с помощью разных функций из URL-адреса https://en.wikipedia.org/wiki/Systems_design.

Таблица 8.2

Хеш-функция	Значение хеша (шестнадцатеричное)
CRC32	5cb54054
MD5	5a62509a84df9ee03fe1230b9df8b84e
SHA-1	0eeae7916c06853901d9ccbefbfcaf4de57ed85b

Как видно из табл. 8.2, даже самое короткое значение хеша (из CRC32) получается слишком большим (больше 7 символов). Как его сократить?

В качестве одного из решений можно взять первые 7 символов хеша, но это чревато конфликтами. Чтобы значения хеша не дублировались, мы можем рекурсивно добавлять к ним заданную строку, пока они не станут уникальными. Этот процесс проиллюстрирован на рис. 8.5.

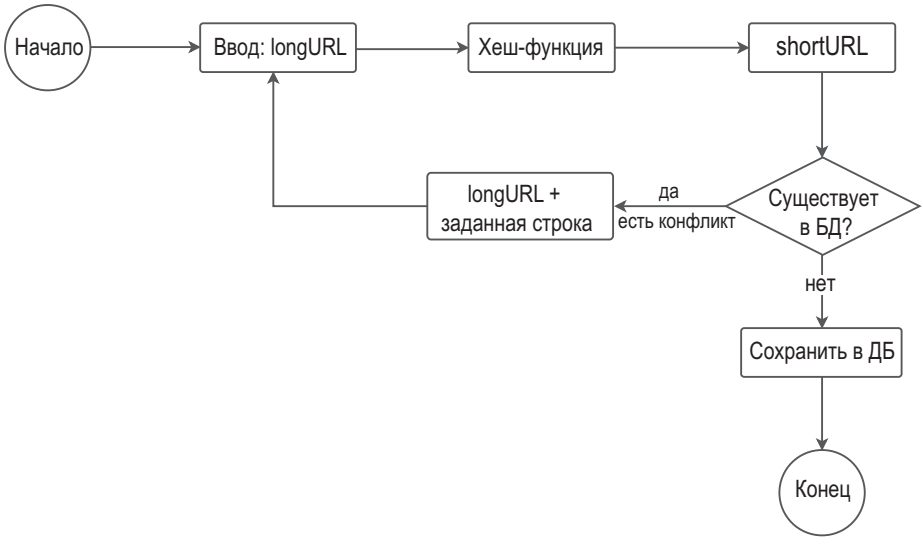


Рис. 8.5

Этот метод может избавить нас от конфликтов, но обращаться к базе данных при каждом запросе, чтобы проверить наличие в ней shortURL, довольно расточительно. Для улучшения производительности можно использовать фильтр Блума [2]. Это вероятностная методика с эффективным использованием пространства, которая позволяет проверить, входит ли элемент в множество. Подробнее об этом читайте в справочных материалах [2].

Преобразование base62

Еще одним распространенным методом сокращения URL-адресов является преобразование значения в другую систему счисления. Поскольку для `hashValue` используется набор из 62 символов, мы выберем алгоритм base62. Чтобы понять, как происходит это преобразование, переведем десятичное число 11157_{10} в вид с основанием 62.

- Как понятно из названия, base62 — это способ кодирования с использованием 62 символов. Они соотносятся как 0-0, ..., 9-9, 10-a, 11-b, ..., 35-z, 36-A, ..., 61-Z, где «a» соответствует 10, «Z» соответствует 61 и т. д.
- $11157_{10} = 2 \times 62^2 + 55 \times 62^1 + 59 \times 62^0 = [2, 55, 59] \rightarrow [2, T, X]$ в представлении base62. Процесс преобразования показан на рис. 8.6.

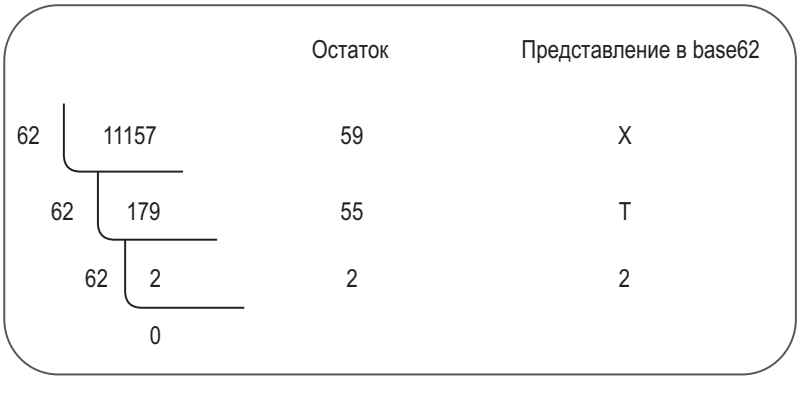


Рис. 8.6

- Таким образом, сокращенный URL-адрес выглядит как https://tinyurl.com/2TX.

Сравнение двух подходов

В табл. 8.3 приводятся отличия этих двух подходов.

Таблица 8.3

Хеш + разрешение конфликтов	Преобразование base62
Фиксированная длина сокращенного URL-адреса	Сокращенный URL-адрес имеет переменную длину, которая увеличивается вместе с ID
Не требует генератора уникальных ID	Этот подход использует генератор уникальных ID
Возможны конфликты, которые нужно разрешать	Конфликты исключены, так как ID уникальные
Невозможно определить, каким будет следующий сокращенный URL-адрес, так как он не зависит от ID	Если ID всегда инкрементируется на 1, можно легко определить следующий доступный URL-адрес. Это может стать угрозой безопасности

Тщательный анализ сокращения URL-адресов

Процесс сокращения URL-адресов является одним из ключевых элементов системы, поэтому мы хотим сделать его простым и функциональным. В нашей архитектуре используется преобразование base62. Принцип работы показан на следующей диаграмме (рис. 8.7).

1. longURL подается на вход.
2. Система проверяет, есть ли longURL в базе данных.
3. Если да, это означает, что значение longURL уже было преобразовано в shortURL. В этом случае мы извлекаем shortURL из базы данных и возвращаем его клиенту.
4. Если нет, longURL является новым адресом и для него генерируется новый уникальный ID (первичный ключ).
5. Преобразуем ID в shortURL методом base62.
6. Создаем в БД новую запись с ID, shortURL и longURL.

Чтобы вам было проще понять этот процесс, рассмотрим конкретный пример.

- Предположим, longURL имеет значение https://en.wikipedia.org/wiki/Systems_design.

- Генератор уникальных ID возвращает 2009215674938.
- Преобразуем ID в shortURL методом base62. 2009215674938 превращается в zn9edcu.
- Сохраняем ID, shortURL и longURL в базу данных, как показано в табл. 8.4.

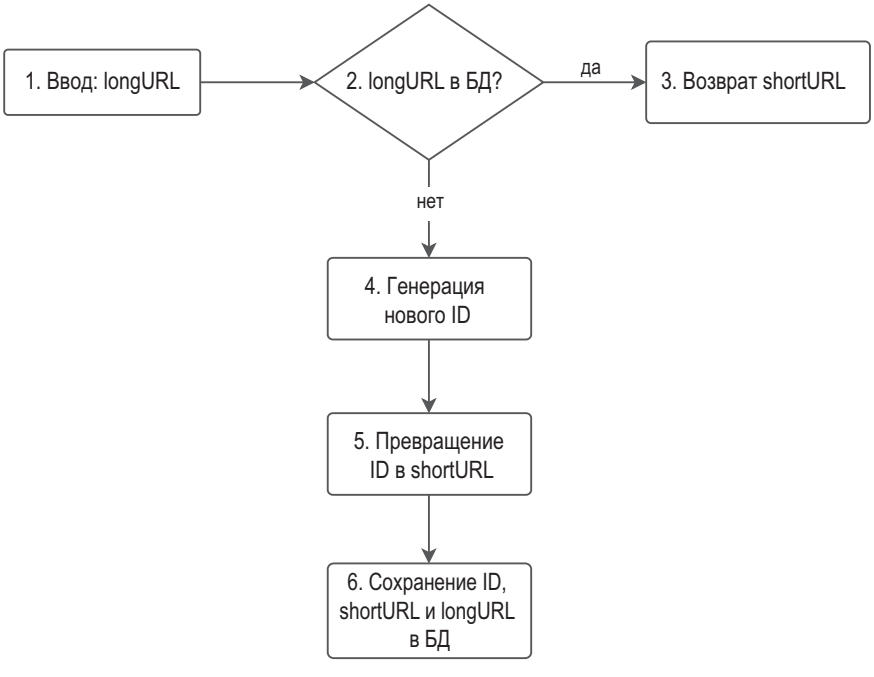


Рис. 8.7

Таблица 8.4

id	shortURL	longURL
2009215674938	zn9edcu	https://en.wikipedia.org/wiki/Systems_design

Стоит упомянуть о распределенном генераторе уникальных ID. Его основная задача состоит в получении идентификаторов, которые используются для создания значений `shortURL` и не повторяются в рамках всей

системы. Реализация генератора уникальных ID в сильно распределенном окружении является непростой задачей. К счастью, мы уже обсудили несколько решений в главе 7 «Проектирование генератора уникальных идентификаторов в распределенных системах». Перечитайте ее, если хотите освежить память.

Тщательный анализ перенаправления URL-адресов

На рис. 8.8 показана подробная схема перенаправления URL-адресов. Поскольку чтение происходит чаще, чем запись, пара `<shortURL, longURL>` хранится в кэше для улучшения производительности.

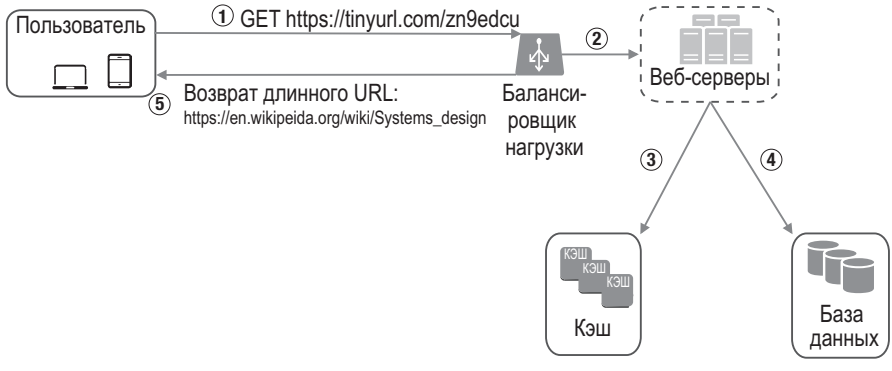


Рис. 8.8

Ниже кратко изложен процесс перенаправления URL-адресов:

1. Пользователь кликает по короткой ссылке https://tinyurl.com/zn9edcu.
2. Балансировщик нагрузки направляет запрос к веб-серверам.
3. Если `shortURL` уже есть в кэше, то сразу возвращается `longURL`.
4. Если `shortURL` нет в кэше, то `longURL` извлекается из базы данных. Если этого адреса нет в БД, пользователь, скорее всего, ввел его неправильно.
5. Пользователю возвращается `longURL`.

ШАГ 4: ПОДВЕДЕНИЕ ИТОГОВ

В этой главе мы поговорили о проектировании API, модели данных, функции хеширования, а также о сокращении и перенаправлении URL-адресов.

Если в конце интервью остается время, можно обсудить дополнительные вопросы.

- Ограничитель трафика. Мы можем столкнуться с потенциальной проблемой безопасности: злоумышленники могут послать чрезмерно большое количество запросов на сокращение URL-адреса. Ограничитель трафика помогает фильтровать запросы с учетом IP-адреса или других правил. Если вам нужно вспомнить эту тему, вернитесь к главе 4 «Проектирование ограничителя трафика».
- Масштабирование веб-серверов. Поскольку веб-уровень не хранит свое состояние, его легко масштабировать, добавляя или удаляя веб-серверы.
- Масштабирование базы данных. Репликация и сегментирование базы данных являются распространенными методиками.
- Аналитика. Данные играют все более важную роль в успехе бизнеса. Интеграция аналитической системы в сервис для сокращения URL-адресов поможет получить такие ценные сведения, как количество пользователей, перешедших по ссылке, точное время перехода и т. д.
- Доступность, согласованность и надежность. Эти характеристики являются ключом к успеху любой крупной системы. Мы подробно обсуждали их в главе 1. Пожалуйста, вспомните эту тему.

Поздравляем, вы проделали длинный путь и можете собой гордиться. Отличная работа!

СПРАВОЧНЫЕ МАТЕРИАЛЫ

[1] Руководство по REST: https://www.restapitutorial.com/index.html

[2] Фильтр Блума: https://ru.wikipedia.org/wiki/Фильтр_Блума